

CSA4604 — MERN Stack Lab

Observation Notes — Experiments 1 to 7

Experiment 1: Git & GitHub Basics

Aim: Installation and configuration of Git; repository creation, commit history, and remote repository management.

Observations:

- Installed and configured Git on the system.
- Initialized a local repository.
- Staged and committed files to the local repository.
- Created a repository on GitHub.
- Connected the local repository to the GitHub repository.
- Pushed the project from local to GitHub.

Commands used:

```
git init
git add .
git commit -m "Initial commit"
git remote add origin <repository-url>
git push -u origin main
```

Experiment 2: Node.js, npm & Simple Express Application

Aim: Installation of Node.js and npm; creation of a simple Node.js application and dependency management using npm.

Observations:

- Created a new Node.js project using `npm init -y`.
- Installed Express using `npm install express`.
- Enabled ES6 module syntax by adding `"type": "module"` in `package.json`.
- Created and ran a basic Express server.

Code:

```
// package.json (relevant part)
{
  "type": "module"
}
```

```
// index.js
import express from 'express';

const app = express();
```

```
app.get('/', (req, res) => {
  res.send('Hello World');
});

app.listen(3000, () => {
  console.log('Server running on port 3000');
});
```

Experiment 3: Handling HTTP Requests and Responses in Node.js

Aim: Implementation of basic HTTP server functionalities — handling requests and responses in Node.js.

Observations:

- Created multiple `GET` routes using Express.
- Used `req.query` to read query string parameters from the URL.
- Filtered data from an in-memory array based on the query parameter.
- Created a basic web server using Node's native `http` module for comparison.

Code (Express — query strings):

```
// index.js
const movies = [
  { title: 'Movie A', genre: 'Sci-Fi' },
  { title: 'Movie B', genre: 'Romance' }
];

app.get('/search', (req, res) => {
  const genre = req.query.genre;
  const results = movies.filter(m => m.genre === genre);
  res.send(results);
});
```

Code (Native Node.js `http` module):

```
// index.js
import http from 'http';

const server = http.createServer((req, res) => {
  if (req.url === '/') {
    res.end('Home Page');
  } else if (req.url === '/about') {
    res.end('About Page');
  } else {
    res.end('Page Not Found');
  }
});

server.listen(3000, () => {
  console.log('Server running on port 3000');
});
```

Experiment 4: Building CRUD Routes in Node.js Applications

Aim: Running and testing Node.js applications — creating applications using installed modules and handling CRUD operations.

Observations:

- Used the `express.json()` middleware to parse incoming JSON request bodies.
- Wrote `GET`, `POST`, `PUT`, and `DELETE` routes to perform CRUD operations on an in-memory array.
- Tested each route to create, read, update, and delete items.

Code:

```
// index.js
app.use(express.json());

let items = [];
let nextId = 1;

// GET one item
app.get('/items/:id', (req, res) => {
  const item = items.find(i => i.id === Number(req.params.id));
  res.send(item);
});

// POST - create item
app.post('/items', (req, res) => {
  const newItem = { id: nextId++, ...req.body };
  items.push(newItem);
  res.send(newItem);
});

// PUT - update item
app.put('/items/:id', (req, res) => {
  const item = items.find(i => i.id === Number(req.params.id));
  Object.assign(item, req.body);
  res.send(item);
});

// DELETE - remove item
app.delete('/items/:id', (req, res) => {
  const index = items.findIndex(i => i.id === Number(req.params.id));
  const [deleted] = items.splice(index, 1);
  res.send(deleted);
});
```

Experiment 5: File Handling in Node.js

Aim: File handling in Node.js using the `fs` module.

Observations:

- Used `fs.readFile()` to read the contents of a file.
- Used `fs.writeFile()` to write data to a file.
- Used `fs.appendFile()` to append data to an existing file.

Code:

```
// index.js
import fs from 'fs';

// Read a file
fs.readFile('data.txt', 'utf8', (err, data) => {
  console.log(data);
});

// Write to a file (overwrites existing content)
fs.writeFile('data.txt', 'Hello World', (err) => {
  console.log('File written');
});

// Append to a file
fs.appendFile('data.txt', '\nNew Line', (err) => {
  console.log('Content appended');
});
```

Experiment 6: Creating a Node.js Server & Processing Client Requests with Middleware

Aim: Creating a Node.js server and processing client requests with appropriate responses using middleware.

Observations:

- Served static files (CSS/JS/images) using the `express.static()` middleware.
- Understood middleware execution order using `app.use()`.
- Used `next()` to pass control from one middleware function to the next.
- Chained multiple middleware functions on a single route before sending a response.

Code:

```
// index.js
app.use(express.static('public'));

function validateGenre(req, res, next) {
  if (!req.query.genre) {
    return res.send('Genre is required');
  }
  next();
}

function handleSearch(req, res) {
  res.send(`Searching for genre: ${req.query.genre}`);
}
```

```
app.get('/search', validateGenre, handleSearch);
```

Experiment 7: Custom Modules & File Operations using Node.js FS Module

Aim: Development of custom modules and file operations using the Node.js `fs` module.

Observations:

- Split reusable code across separate files (custom modules).
- Used **named exports** and imported them using named imports.
- Used a **default export** (a bundled object of functions) and imported it using a default import.
- Used a **namespace import** to import all named exports from a file at once.
- Combined a default export and named exports in the same file and imported both together.
- Reused `fs` file operations (`readFileSync`, `writeFileSync`) inside a custom module.

Code:

```
// mathUtils.js - named exports
export const add = (a, b) => a + b;
export const subtract = (a, b) => a - b;
export const PI = 3.14;
```

```
// stringUtils.js - default export (bundled object)
const upper = (str) => str.toUpperCase();
const lower = (str) => str.toLowerCase();
const reverse = (str) => str.split('').reverse().join('');

export default { upper, lower, reverse };
```

```
// arrayUtils.js - named exports (imported as namespace)
export const first = (arr) => arr[0];
export const last = (arr) => arr[arr.length - 1];
export const sum = (arr) => arr.reduce((a, b) => a + b, 0);
```

```
// fileUtils.js - mixed default + named exports
import fs from 'fs';

export default function greet() {
  console.log('Hello from fileUtils');
}

export function readData(path) {
  return fs.readFileSync(path, 'utf8');
}

export function saveData(path, data) {
  fs.writeFileSync(path, data);
}
```

```
// index.js - importing all four modules
import { add, subtract, PI } from './mathUtils.js';
```

```
import stringUtils from './stringUtils.js';
import * as arrayUtils from './arrayUtils.js';
import greet, { readData, saveData } from './fileUtils.js';

console.log(add(2, 3));
console.log(stringUtils.upper('hello'));
console.log(arrayUtils.sum([1, 2, 3]));
greet();
```
